



CRC 2026

Java w praktyce – tworzenie i zarządzanie aplikacjami

Streamy i kolekcje w Javie

ING Hubs Poland Kamil Jędrychowski

April 2026



# Agenda

- Typy generyczne w Javie
- Ćwiczenia
- Kolekcje w Javie
- Ćwiczenia
- Streamy w Javie
- Ćwiczenia
- Podsumowanie

# Typy generyczne w Javie

# Problem przed typami generycznymi

- Kolekcje przechowywały obiekty typu Object
- Zawsze potrzebne było rzutowanie – brakowało bezpieczeństwa typów i kod był mniej czytelny

```
List list = new ArrayList();  
list.add("tekst");  
String s = (String) list.get(0); // rzutowanie
```

# Podstawy

- Typy generyczne pozwalają nam określić typ danych
- Przykłady: `List<String>`, `Map<String, String>`

```
1 List<String> names = new ArrayList<>();  
2 names.add("Ala");  
3 // names.add(10); // błąd kompilacji
```

# Własne klasy generyczne

- Deklarujemy parameter typu: <T>
- Przykłady: List<String>, Map<String, String>
- Box<Integer> box = new Box<>();

```
class Box<T> {  
    private T value;  
  
    public void set(T value) {  
        this.value = value;  
    }  
  
    public T get() {  
        return value;  
    }  
}
```

# Ograniczenia i wildcards

```
class Calculator<T extends Number> { }
```

- <?> – dowolny typ
- <? extends Number> – do odczytu
- <? super Integer> – do zapisu

```
1 List<? extends Number> numbers;
```

# Zadania



# Zadania - streamy

Zaimplementuj klasę:

```
1 Pair<K, V>
```

Napisz metodę:

```
1 public static <T> boolean isEqual(T a, T b)
```

# Kolekcje w Javie

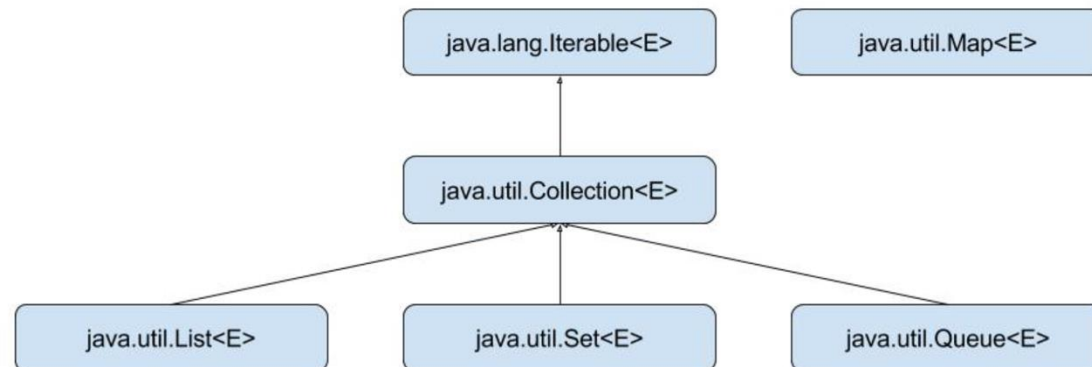
# Czym są kolekcje w Javie

**Kolekcje** to zorganizowana grupa obiektów. Pozwalają na przechowywanie grupy obiektów w określony sposób, a także manipulowanie nimi.

Są bardzo przydatne do zarządzania dużymi ilościami danych oraz zawierają metody, które pozwalają nam na dodawanie, usuwanie, wyszukiwanie i sortowanie elementów.

W Javie wszystkie kolekcje znajdują się w pakiecie `java.util.*`

Podstawowe kolekcje to: List, Set, Queue, Map.

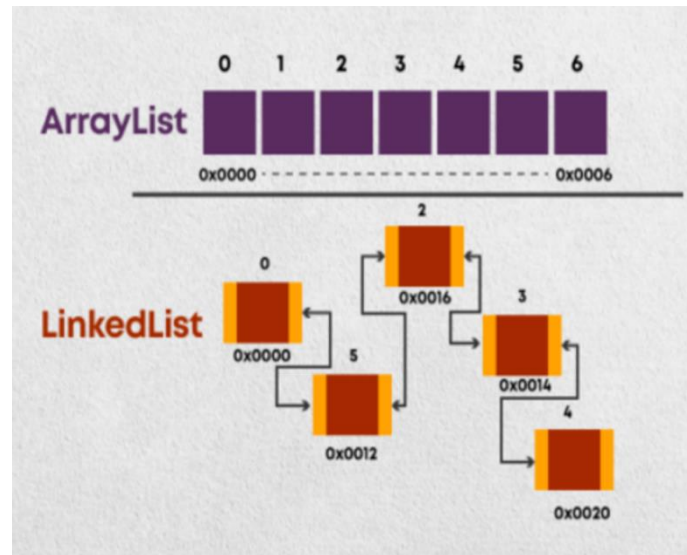


# List

**Lista** przechowuje element w określonej kolejności.

Podstawowe implementacje listy to:

- ArrayList – Dynamiczna tablica, która daje nam szybki dostęp do elementów.
- LinkedList – Lista dwukierunkowa, która pozwala na szybkie operacje dodawania i usuwania.



# List

Podstawowe metody które zawiera List:

- `add(E e)`: Dodaje element do listy.
- `get(int index)`: Pobiera element z określonego indeksu.
- `remove(int index)`: Usuwa element z określonego indeksu.
- `size()`: Zwraca liczbę elementów w liście.
- `contains(Object o)`: Sprawdza, czy lista zawiera określony element.
- `clear()`: Usuwa wszystkie elementy z listy.
- `isEmpty()`: Sprawdza, czy lista jest pusta.
- `indexOf(Object o)`: Zwraca indeks pierwszego wystąpienia określonego elementu.

# Set

**Set** to taki zbiór danych, który zawiera unikalne element bez określonej kolejności.

Podstawowe implementacje Set to:

- HashSet – który pozwala na szybkie dodawanie I usuwanie elementów bez zachowania kolejności
- TreeSet - który zachowuje kolejność, jednak operacje na nim są wolniejsze

| Parameters                               | HashSet    | TreeSet        |
|------------------------------------------|------------|----------------|
| Data Structure                           | Hash Table | Red-black Tree |
| Time Complexity<br>(add/remove/contains) | $O(1)$     | $O(\log n)$    |
| Iteration Order                          | Arbitrary  | Sorted         |
| Null Values                              | Allowed    | Not Allowed    |
| Processing                               | Fast       | Slow           |

HashSet Vs TreeSet

# Set

## Podstawowe metody które zawiera Set:

- `add(E e)`: Dodaje element do zestawu.
- `remove(Object o)`: Usuwa określony element z zestawu.
- `contains(Object o)`: Sprawdza, czy zestaw zawiera określony element.
- `size()`: Zwraca liczbę elementów w zestawie.
- `clear()`: Usuwa wszystkie elementy z zestawu.
- `isEmpty()`: Sprawdza, czy zestaw jest pusty.
- `iterator()`: Zwraca iterator do iteracji przez elementy zestawu.

# Queue

**Queue** to kolekcja która przechowuje element w kolejności FIFO lub FILO.

Podstawowe implementacje kolejki to:

- Linked List, która może być wykorzystywana jako kolejka
- PriorityQueue – w której element są sortowane według naturalnego porządku lub komparatora, który może ustalać priorytet danego element.

Podstawowe metody które zawiera kolejka to:

- `add(E e)`: Dodaje element do kolejki.
- `offer(E e)`: Dodaje element do kolejki, zwraca `true` jeśli się udało.
- `poll()`: Pobiera i usuwa element z początku kolejki.
- `peek()`: Pobiera element z początku kolejki bez usuwania.
- `remove(Object o)`: Usuwa określony element z kolejki.
- `size()`: Zwraca liczbę elementów w kolejce.
- `isEmpty()`: Sprawdza, czy kolejka jest pusta.
- `iterator()`: Zwraca iterator do iteracji przez elementy kolejki.



# Map

**Map** to kolekcja która przechowuje pary klucz-wartość.

Podstawowe implementacje mapy to:

- HashMap – która pozwala na szybkie operacje i nie utrzymuje kolejności elementów
- TreeMap – która przechowuje posortowane klucze, jednak operacje na niej są wolniejsze

| Property \ Map                   | HashMap        | TreeMap                          |
|----------------------------------|----------------|----------------------------------|
| Ordering                         | not guaranteed | sorted,<br>natural ordering      |
| get / put / remove<br>complexity | $O(1)$         | $O(\log(n))$                     |
| Inherited interfaces             | Map            | Map<br>NavigableMap<br>SortedMap |
| NULL values / keys               | allowed        | only values                      |

# Map

Podstawowe metody, które zawierają mapy to:

- `put(K key, V value)`: Dodaje parę klucz-wartość do mapy.
- `get(Object key)`: Pobiera wartość na podstawie klucza.
- `remove(Object key)`: Usuwa parę klucz-wartość na podstawie klucza.
- `containsKey(Object key)`: Sprawdza, czy mapa zawiera określony klucz.
- `containsValue(Object value)`: Sprawdza, czy mapa zawiera określoną wartość.
- `size()`: Zwraca liczbę par klucz-wartość w mapie.
- `clear()`: Usuwa wszystkie pary klucz-wartość z mapy.
- `keySet()`: Zwraca zestaw kluczy w mapie.
- `values()`: Zwraca kolekcję wartości w mapie.

# Porównanie podstawowych kolekcji

| Typ Kolekcji | Kolejność    | Unikalność   | Implementacje             |
|--------------|--------------|--------------|---------------------------|
| List         | Tak          | Nie          | ArrayList, LinkedList     |
| Set          | Nie          | Tak          | HashSet, TreeSet          |
| Queue        | Tak (FIFO)   | Nie          | LinkedList, PriorityQueue |
| Map          | Tak (klucze) | Tak (klucze) | HashMap, TreeMap          |

**Dwie ważne metody, które wykorzystywane są m.in. w kolekcjach**

# Metoda equals

Celem metody equals jest porównanie dwóch obiektów pod względem równości. Domyślna implementacja porównuje referencje obiektów (czy wskazują na ten sam obiekt).

```
@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (obj == null || getClass() != obj.getClass()) return false;
    MyClass myClass = (MyClass) obj;
    return Objects.equals(field1, myClass.field1) &&
        Objects.equals(field2, myClass.field2);
}
```

- **Zastosowanie w kolekcjach:**
  - **List:** Sprawdza, czy lista zawiera określony element (contains).
  - **Set:** Sprawdza, czy zestaw zawiera unikalne elementy.
  - **Map:** Sprawdza, czy mapa zawiera określony klucz (containsKey).

# Metoda hashCode

Celem metody hashCode jest generowanie liczby reprezentującej obiekt.

```
@Override
public int hashCode() {
    return Objects.hash(field1, field2);
}
```

- **Zastosowanie w kolekcjach:**
  - **HashSet:** Używa hashCode do szybkiego wyszukiwania elementów.
  - **HashMap:** Używa hashCode kluczy do szybkiego dostępu do wartości.
  - **HashTable:** Używa hashCode do organizacji danych w tablicach.

# Zasady implementacji metod equals i hashCode.

## Przykłady zastosowania

- equals:

- Symetryczność: `a.equals(b)` powinno być równe `b.equals(a)`.
- Transitivity: Jeśli `a.equals(b)` i `b.equals(c)`, to `a.equals(c)`.
- Konsystencja: Wynik `equals` powinien być spójny, jeśli obiekty się nie zmieniają.

- hashCode:

- Konsystencja: Wynik `hashCode` powinien być spójny, jeśli obiekt się nie zmienia.
- Równość: Jeśli `a.equals(b)`, to `a.hashCode() == b.hashCode()`.

- HashSet:

```
Set<MyClass> set = new HashSet<>();  
set.add(new MyClass("A", 1));  
set.add(new MyClass("B", 2));
```

- HashMap:

```
Map<MyClass, String> map = new HashMap<>();  
map.put(new MyClass("A", 1), "Value1");  
map.put(new MyClass("B", 2), "Value2");
```

## Zadania - 1



# Zadania - kolekcje

1. Utwórz `ArrayList` i dodaj 5 elementów.
2. Usuń element z `LinkedList`.
3. Sprawdź, czy `HashSet` zawiera określony element.
4. Posortuj `TreeSet`.
5. Dodaj element do `PriorityQueue`.
6. Pobierz wartość z `HashMap` na podstawie klucza.
7. Iteruj przez elementy `ArrayList`.
8. Iteruj przez elementy `HashSet`.
9. Iteruj przez elementy `PriorityQueue`.
10. Iteruj przez klucze `TreeMap`.
11. Utwórz `LinkedList` jako kolejkę.
12. Usuń element z `PriorityQueue`.
13. Sprawdź, czy `TreeSet` jest pusty.
14. Pobierz klucz z `HashMap`.
15. Dodaj element do `LinkedList`.
16. Usuń element z `ArrayList`.
17. Sprawdź rozmiar `HashSet`.
18. Sprawdź rozmiar `TreeMap`.
19. Pobierz pierwszy element z `PriorityQueue`.
20. Pobierz ostatni element z `LinkedList`.

# Zadania - kolekcje

1. Napisz program, który będzie pobierał od użytkownika imiona. Program powinien pozwolić użytkownikowi na wprowadzenie dowolnej liczby imion (wprowadzenie „-” jako imienia przerwie wprowadzanie). Na zakończenie wypisz liczbę unikalnych imion.
2. Napisz program, który będzie pobierał od użytkownika imiona par dopóki nie wprowadzi imienia „-”, następnie poproś użytkownika o podanie jednego z wcześniej wprowadzonych imion i wyświetl imię odpowiadającego mu partnera.

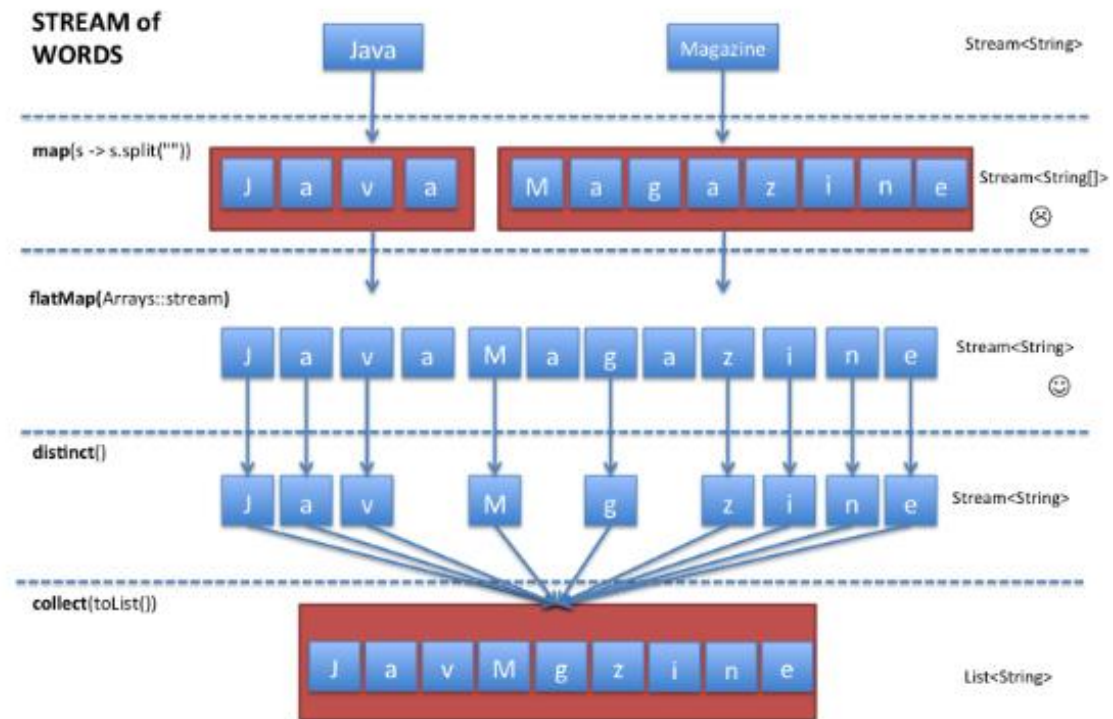
# Streamy w Javie

# Czym są Streamy w Javie

**Stream** to sekwencje elementów wspierające operacje agregujące.  
Źródłem streamów mogą być kolekcje, tablice, funkcje generatorów lub kanały I/O.

Przykładowe rodzaje to:

- Stream<T>
- IntStream
- LongStream
- DoubleStream



# Przykłady tworzenia Streamów

- Z kolekcji:

```
List<String> list = Arrays.asList("a", "b", "c");  
Stream<String> stream = list.stream();
```

- Z tablicy:

```
String[] array = {"a", "b", "c"};  
Stream<String> stream = Arrays.stream(array);
```

- Z wartości:

```
Stream<String> stream = Stream.of("a", "b", "c");
```

- Z funkcji generatora:

```
Stream<Double> stream = Stream.generate(Math::random).limit(10);
```

# Przykładowe operacje pośrednie

- **filter(Predicate)**: Filtruje elementy.

```
stream.filter(s -> s.startsWith("a"));
```

- **map(Function)**: Przekształca elementy.

```
stream.map(String::toUpperCase);
```

- **flatMap(Function)**: Przekształca elementy na strumienie i łączy je.

```
stream.flatMap(s -> Stream.of(s.split("")));
```

- **distinct()**: Usuwa duplikaty.

```
stream.distinct();
```

- **sorted()**: Sortuje elementy.

```
stream.sorted();
```

- **peek(Consumer)**: Wykonuje akcję dla każdego elementu.

```
stream.peek(System.out::println);
```

- **limit(long)**: Ogranicza liczbę elementów.

```
stream.limit(5);
```

- **skip(long)**: Pomija pierwsze n elementów.

```
stream.skip(3);
```

# Przykładowe operacje końcowe

- **forEach(Consumer)**: Wykonuje akcję dla każdego elementu.

```
stream.forEach(System.out::println);
```

- **toArray()**: Zwraca tablicę.

```
String[] array = stream.toArray(String[]::new);
```

- **reduce(BinaryOperator)**: Redukuje elementy do jednej wartości.

```
Optional<String> result = stream.reduce((s1, s2) -> s1 + s2);
```

- **collect(Collector)**: Zbiera elementy do kolekcji.

```
List<String> list = stream.collect(Collectors.toList());
```

- **min(Comparator)**: Zwraca minimalny element.

```
Optional<String> min = stream.min(String::compareTo);
```

- **max(Comparator)**: Zwraca maksymalny element.

```
Optional<String> max = stream.max(String::compareTo);
```

- **count()**: Zwraca liczbę elementów.

```
long count = stream.count();
```

- **anyMatch(Predicate)**: Sprawdza, czy jakikolwiek element spełnia warunek.

```
boolean anyMatch = stream.anyMatch(s -> s.startsWith("a"));
```

- **allMatch(Predicate)**: Sprawdza, czy wszystkie elementy spełniają warunek.

```
boolean allMatch = stream.allMatch(s -> s.length() > 1);
```

- **noneMatch(Predicate)**: Sprawdza, czy żaden element nie spełnia warunku.

```
boolean noneMatch = stream.noneMatch(s -> s.isEmpty());
```

- **findFirst()**: Zwraca pierwszy element.

```
Optional<String> first = stream.findFirst();
```

- **findAny()**: Zwraca dowolny element.

```
Optional<String> any = stream.findAny();
```

## Zadania - 2



# Zadania - streamy

1. Utwórz listę liczb całkowitych i przefiltruj liczby parzyste, a następnie zbierz je do nowej listy.
2. Utwórz listę imion i przekształć je na wielkie litery.
3. Utwórz listę liczb całkowitych i posortuj je w kolejności malejącej.
4. Utwórz listę liczb całkowitych i oblicz ich sumę za pomocą metody reduce.
5. Utwórz listę imion i policz, ile z nich zaczyna się na literę "A".
6. Utwórz listę liczb całkowitych i znajdź pierwszy element większy niż 10.
7. Utwórz listę liczb całkowitych i znajdź najmniejszą wartość.
8. Utwórz listę liczb całkowitych i znajdź największą wartość.
9. Utwórz listę liczb całkowitych i sprawdź, czy wszystkie liczby są dodatnie.
10. Utwórz listę liczb całkowitych i sprawdź, czy jakakolwiek liczba jest ujemna.
11. Utwórz listę imion i przekształć ją na tablicę.
12. Utwórz listę imion i pogrupuj je według pierwszej litery.
13. Utwórz dwie listy liczb całkowitych i połącz je w jeden strumień.
14. Utwórz listę list liczb całkowitych i przekształć ją na pojedynczy strumień liczb.
15. Utwórz listę liczb całkowitych i ogranicz strumień do pierwszych 5 elementów.
16. Utwórz listę liczb całkowitych i pomiń pierwsze 3 elementy.
17. Utwórz listę imion i wypisz każde imię na konsolę.
18. Utwórz tablicę liczb całkowitych i przekształć ją na strumień.
19. Utwórz strumień z wartości "a", "b", "c".
20. Utwórz strumień 10 losowych liczb.

**Dodatkowe zadanie**

# Zadania

1. Przygotuj klasę która będzie zawierała kilka pól m.in. String, Integer..
2. Zaimplementuj metody equals I hashCode.
3. Utwórz kilka kolekcji które zawierają tą klasę.
4. Porównaj dwa takie same obiekty tej klasy.
5. Porównaj wynik metody hashCode dwóch obiektów tej klasy



do your thing